

GUIDE DE DÉMONSTRATION LIVE — BIPROJECT

1. Objectif du Document

Ce guide a pour but de fournir une marche à suivre rigoureuse et simple pour lancer le projet **BiProject** et réaliser une démonstration fluide devant un professeur. Suivez les étapes dans l'ordre pour éviter toute erreur technique en direct.

2. Prérequis Systèmes

Assurez-vous que les outils suivants sont installés sur le poste de présentation :

- **.NET 8 SDK** (Pour le Backend)
 - **Node.js & npm** (Pour le Frontend)
 - **Angular CLI** (`npm install -g @angular/cli`)
 - **Python 3.x & pip** (Pour les tests Selenium)
 - **Google Chrome** (Navigateur de référence)
-

3. Lancement du Projet (Ordre de marche)

Préparez **deux terminaux PowerShell** distincts.

Étape A : Lancement du Backend (Terminal 1)

```
cd BiProject.Api  
dotnet run
```

Attendre le message : *Now listening on: http://localhost:5120*

Étape B : Lancement du Frontend (Terminal 2)

```
cd BiProject.Ui  
npm start
```

Attendre le message : *Compiled successfully.*

4. URLs de Vérification

Une fois les serveurs lancés, ouvrez Chrome sur :

- **Application Web** : <http://localhost:4200/login>
 - **Swagger API** (Optionnel) : <http://localhost:5120/swagger>
-

5. Vérification Manuelle Rapide (Échauffement)

Avant d'appeler le professeur, testez ces deux points :

1. **Connexion** : Saisissez `test_user` / `TestPassword123!`. Vous devez arriver sur `/products`.
2. **AuthGuard** : Déconnectez-vous, puis essayez d'accéder directement à `http://localhost:4200/products`.
Le système doit vous renvoyer vers `/login`.

6. Lancement des Tests (La preuve technique)

Ouvrez un **troisième terminal** pour les commandes suivantes :

Tests Backend (Unitaires & Intégration)

```
dotnet test BiProject.Tests
```

Le rapport HTML horodaté est généré automatiquement dans `BiProject.Tests/TestResults/`.

Tests Système (Selenium)

```
# S'assurer d'être à la racine du projet  
py -m pytest tests_selenium/tests -v
```

Le rapport HTML horodaté est généré automatiquement dans `tests_selenium/results/`.

Tests Robot Framework (Bonus)

```
# S'assurer d'être à la racine du projet  
py -m robot --outputdir tests_robot/results tests_robot/login_tests.robot
```

7. Démo devant le Professeur (Scénario suggéré)

Voici l'ordre recommandé pour "vendre" votre projet en 5 minutes :

1. **Présentation de l'UI** : Montrez la page de Login et le Catalogue.
 2. **Sécurité (Static Test)** : Expliquez que vous avez détecté une faille IDOR par analyse statique.
 3. **Preuve par le Test** : Lancez les tests Selenium en direct pour montrer l'automatisation.
 4. **Démonstration de la Faille** : Montrez le test `CT-18` qui échoue (Fail attendu). Expliquez que cela prouve votre capacité à détecter des vulnérabilités critiques de sécurité.
 5. **Conclusion** : Montrez la matrice de traçabilité dans la documentation pour prouver la rigueur de votre démarche.
-

8. Points d'Attention (Éviter les "Effets Démo")

- **Ports** : Vérifiez qu'aucune autre application n'utilise déjà le port **5120** ou **4200**.
 - **Base de Données** : Le projet utilise une base de données en mémoire (ou locale volatile) ; les données sont réinitialisées à chaque lancement du backend.
 - **Focus Navigateur** : Lors des tests Selenium, ne touchez pas à la souris ou au clavier, laissez le script manipuler Chrome.
-

9. Résumé Ultra-Court

Action	Commande
Run Backend	<code>dotnet run</code> (dans BiProject.Api)
Run Frontend	<code>npm start</code> (dans BiProject.Ui)
Tests C#	<code>dotnet test BiProject.Tests</code>
Tests Selenium	<code>py -m pytest</code>
Tests Robot	<code>py -m robot tests_robot/login_tests.robot</code>

Bonne chance pour votre présentation !